VAE-Based Image Inpainting
A Research Survey and Architectural Proposal

FlowerPower Project — Internal Research Document

July 13, 2026

Abstract

The FlowerPower project currently compares two generative approaches for image inpainting on 128×128 flower images: a custom U-Net GAN trained from scratch and a fine-tuned LaMa (Large Mask Inpainting) model based on Fast Fourier Convolutions. This document investigates the feasibility of introducing a third generative paradigm—the Variational Autoencoder (VAE)—to the comparison. We present (i) the mathematical foundations of VAEs and their adaptation to conditional inpainting, (ii) a survey of existing VAE-based models suitable for fine-tuning, (iii) a detailed proposal for a custom convolutional VAE architecture designed to integrate with the project's existing training infrastructure, and (iv) training strategies to address known VAE failure modes such as posterior collapse and blurriness. Our recommendation is a hybrid VAE–GAN approach that leverages the VAE's principled latent space with adversarial sharpening, producing a clean three-way comparison across generative paradigms.

Contents

1 Introduction and Motivation

The FlowerPower project addresses the problem of **image inpainting**—reconstructing missing or corrupted regions of an image given the surrounding context. The project operates on the Oxford 102 Flowers dataset, resized to 128×128 pixels, with large contiguous masks (halves, quadrants, and large random rectangles covering 30–70% of the image).

Two generative approaches are currently implemented:

1. **U-Net GAN** (`GAN_implementation/`): A custom encoder–decoder generator with skip connections, trained adversarially with a PatchGAN-style discriminator.
2. **LaMa Fine-Tuning** (`lama_finetuning/`): Fine-tuning of the state-of-the-art LaMa model, which uses Fast Fourier Convolutions (FFCs) for image-wide receptive fields.

Both approaches share a common infrastructure defined in `shared_utils.py`: the `ImageDataset`, `Discriminator`, `get_large_random_mask`, and `compute_gradient_loss` utilities. Adding a **Variational Autoencoder (VAE)** introduces a fundamentally different generative paradigm based on probabilistic latent-variable modelling, enabling a principled three-way comparison across architectural philosophies.

Key Insight

A VAE offers something neither the GAN nor LaMa provides: a *structured, interpretable latent space* learned via variational inference. This enables probabilistic reasoning about the inpainted output (e.g., sampling multiple plausible completions) and provides a theoretically grounded training objective (the ELBO), which avoids the mode collapse and training instability common in adversarial training.

2 Mathematical Foundations of VAEs

2.1 The Generative Model and Variational Inference

A Variational Autoencoder [1, 2] posits a latent-variable generative model of the form:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}, \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^D$ is the observed data (an image), $\mathbf{z} \in \mathbb{R}^d$ is a latent representation, $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the prior, and $p_\theta(\mathbf{x} \mid \mathbf{z})$ is the decoder (likelihood) parameterised by a neural network with weights θ.

Since the true posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$ is intractable, we introduce an **approximate posterior** (encoder):

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\boldsymbol{\mu}_\phi(\mathbf{x}),\ \mathrm{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x}))\big), \tag{2}$$

where $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}^2_\phi$ are outputs of the encoder network.

2.2 The Evidence Lower Bound (ELBO)

Training maximises the Evidence Lower Bound on the log-likelihood:

$$\log p_\theta(\mathbf{x}) \geq \text{ELBO}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{\text{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z}))}_{\text{Regularisation term}} \tag{3}$$

The **reconstruction term** encourages the decoder to faithfully reproduce the input from its latent code. The **KL divergence term** regularises the latent space to stay close to the standard normal prior, enabling smooth interpolation and meaningful sampling.

2.3 The Reparameterisation Trick

To enable gradient-based optimisation through the stochastic sampling step, the reparameterisation trick [1] expresses the latent sample as a deterministic, differentiable function of the encoder outputs and an auxiliary noise variable:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{4}$$

2.4 Adaptation to Conditional Inpainting

For inpainting, the model must be **conditioned** on the observed (unmasked) context. Given a masked image $\mathbf{x}_{\text{masked}}$ and a binary mask $\mathbf{m}$, we define a **Conditional VAE (CVAE)** [3] whose encoder and decoder both receive the conditioning information:

$$q_\phi(\mathbf{z} \mid \mathbf{x}, \mathbf{c}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}, \mathbf{c}),\ \text{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x}, \mathbf{c}))), \tag{5}$$

$$p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c}) = \text{Decoder}_\theta(\mathbf{z}, \mathbf{c}), \tag{6}$$

where $\mathbf{c} = [\mathbf{x}_{\text{masked}}; \mathbf{m}]$ is the 4-channel conditioning input (identical to the input format already used by the U-Net GAN and LaMa in the FlowerPower project).

Note

During **training**, the encoder sees the *complete* ground-truth image concatenated with the conditioning context, while the decoder receives only the latent code and the conditioning context. During **inference**, the encoder receives only the masked image and context (or the conditioning is passed directly to the decoder), and the decoder produces the inpainted output.

3 Survey of Existing VAE-Based Models for Inpainting

3.1 VAEAC: VAE with Arbitrary Conditioning

VAEAC [4] is the most directly relevant existing model for our inpainting task. It extends the standard CVAE framework to condition on an *arbitrary* subset of observed features, making it a

natural fit for inpainting where the "observed subset" is defined by the mask.

3.1.1 Architecture

VAEAC introduces three key networks:

- A **proposal network** $q_\phi(\mathbf{z} \mid \mathbf{x}, \mathbf{b})$: encodes the full observation $\mathbf{x}$ with knowledge of which features are observed (via binary mask $\mathbf{b}$).
- A **prior network** $p_\psi(\mathbf{z} \mid \mathbf{x}_{\text{obs}}, \mathbf{b})$: produces a *conditional prior* from only the observed features, replacing the fixed $\mathcal{N}(\mathbf{0}, \mathbf{I})$ prior.
- A **generative network** $p_\theta(\mathbf{x}_{\text{miss}} \mid \mathbf{z}, \mathbf{x}_{\text{obs}}, \mathbf{b})$: decodes the missing features from the latent code and observed context.

  The modified ELBO becomes:

$$\mathcal{L}_{\text{VAEAC}} = -\mathbb{E}_{q_\phi}\big[\log p_\theta(\mathbf{x}_{\text{miss}} \mid \mathbf{z}, \mathbf{x}_{\text{obs}}, \mathbf{b})\big] + \text{KL}\big(q_\phi(\mathbf{z} \mid \mathbf{x}, \mathbf{b}) \,\|\, p_\psi(\mathbf{z} \mid \mathbf{x}_{\text{obs}}, \mathbf{b})\big). \tag{7}$$

3.1.2 Suitability for FlowerPower

Table 1: VAEAC suitability assessment for the FlowerPower project.

Criterion	Assessment
Input format	4-channel $[\mathbf{x}_{\text{masked}} \oplus \mathbf{m}]$ — **identical** to existing project format
Pre-trained weights	Not available for flower images; the published model was demonstrated on MNIST and tabular data
Architecture scaling	Published code uses fully-connected layers; requires replacement with convolutional encoder/decoder for 128×128 images
Learned prior	The conditional prior network is a major advantage: at inference time, the prior adapts to the specific observed context rather than defaulting to $\mathcal{N}(\mathbf{0}, \mathbf{I})$
Implementation effort	**Medium**: architecture concept is ready but requires re-implementation for convolutional image data at our resolution

Reference Implementation

Repository: `tigvarts/vaeac` (PyTorch)
Paper: Ivanov et al., "Variational Autoencoder with Arbitrary Conditioning," ICLR 2019

3.2 Stable Diffusion VAE (AutoencoderKL)

The Stable Diffusion pipeline includes a powerful KL-regularised autoencoder (`AutoencoderKL` from HuggingFace `diffusers`) trained on the LAION-5B dataset. It compresses images from pixel space into a $4 \times h/8 \times w/8$ latent space.

Table 2: Stable Diffusion VAE suitability assessment.

Criterion	Assessment
Pre-trained quality	Excellent encoder/decoder trained on billions of images
Designed for inpainting?	**No** — designed as a compression stage for diffusion models, not standalone inpainting
Standalone performance	Using the VAE alone (without the diffusion U-Net) for inpainting yields blurry, semantically incoherent results in masked regions
Adaptation path	Would require latent-space optimisation or masked reconstruction loss engineering; significant departure from the project's direct-generation paradigm
Implementation effort	**High**: requires fundamentally different inference pipeline

Warning

Using the SD VAE alone (without the diffusion backbone) is unconventional and is **not recommended** for this project. It would introduce an unfair comparison: the SD VAE was never designed to operate as a standalone generator, and its poor standalone inpainting results would not reflect the VAE paradigm fairly.

3.3 Summary and Recommendation

Table 3: Comparative summary of existing VAE options.

Model	Pre-trained?	Inpainting Native?	Integration Effort	Recommendation
VAEAC	×	✓	Medium	Consider
SD AutoencoderKL	✓	×	High	Not recommended
Custom ConvVAE	×	✓	**Low**	**Recommended**

Given the project's goals—a fair three-way comparison with maximal code reuse—**we recommend building a custom convolutional VAE–GAN hybrid** that integrates directly with the existing shared utilities. The architectural details follow in Section 4.

4 Proposed Architecture: Convolutional VAE–GAN for Inpainting

4.1 Design Principles

The proposed architecture follows four design principles motivated by the existing codebase:

1. **Same input/output format**: 4-channel input $[\mathbf{x}_{\text{masked}} \oplus \mathbf{m}]$, 3-channel output in $[-1, 1]$ via tanh.
2. **Same composition**: The output is composited as $\mathbf{x}_{\text{comp}} = \mathbf{x}_{\text{masked}} + \hat{\mathbf{x}} \odot \mathbf{m}$.
3. **Same adversarial framework**: Reuses the shared `Discriminator` for adversarial training.
4. **Same evaluation pipeline**: Compatible with the existing `evaluate.py` visualisation code.

4.2 Architecture Diagram

The following diagram illustrates the complete encoder–decoder structure with the latent bottleneck:

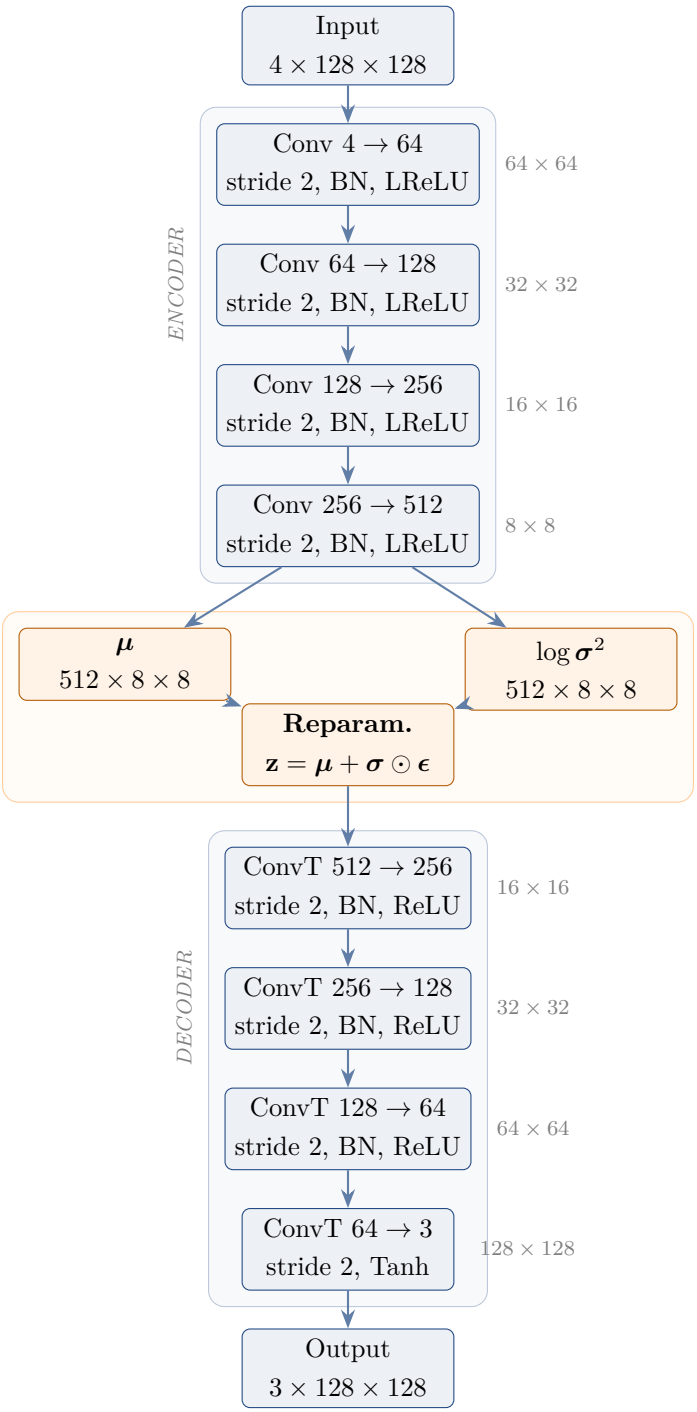

Figure 1: Proposed `ConvVAEGenerator` architecture. The encoder compresses the 4-channel input $[\mathbf{x}_{\text{masked}} \oplus \mathbf{m}]$ into a spatial latent map at 8×8 resolution. The reparameterisation trick samples from the learned distribution, and the decoder reconstructs the 3-channel output. Unlike a flat latent vector, retaining spatial dimensions in the latent space preserves structural information crucial for inpainting.

4.3 Spatial vs. Flat Latent Space

A critical design decision is whether the latent representation should be a flat vector or retain spatial dimensions.

Table 4: Comparison of latent space geometries for inpainting.

	Flat Latent Vector	**Spatial Latent Map**
Shape	d-dimensional vector	$C \times H' \times W'$ tensor
Pros	Maximally compressed; strong regularisation; good for sampling	Preserves spatial structure; easier for the decoder to localise features
Cons	Loses spatial correspondence; struggles with large images	Higher-dimensional; weaker compression
Best for	Generation from scratch	**Inpainting** (needs spatial awareness)

Key Insight

For inpainting tasks, we recommend using a **spatial latent map** of shape $512 \times 8 \times 8 = 32{,}768$ dimensions. This preserves the spatial correspondence between input and output that is critical for large-mask completion, while still providing meaningful compression from the input dimensionality of $4 \times 128 \times 128 = 65{,}536$.

4.4 Detailed Layer Specification

Table 5 provides the complete layer-by-layer specification.

Table 5: Detailed layer specification for the proposed `ConvVAEGenerator`.

Stage	**Layer**	**Configuration**	**Output Shape**	**Parameters**
Encoder	Conv2d + BN + LeakyReLU	$4 \to 64$, k=4, s=2, p=1	$64 \times 64 \times 64$	4,224
	Conv2d + BN + LeakyReLU	$64 \to 128$, k=4, s=2, p=1	$128 \times 32 \times 32$	131,328
	Conv2d + BN + LeakyReLU	$128 \to 256$, k=4, s=2, p=1	$256 \times 16 \times 16$	524,800
	Conv2d + BN + LeakyReLU	$256 \to 512$, k=4, s=2, p=1	$512 \times 8 \times 8$	2,098,176
Lat.	Conv2d ($\boldsymbol{\mu}$)	$512 \to 512$, k=3, s=1, p=1	$512 \times 8 \times 8$	2,359,808
	Conv2d ($\log \boldsymbol{\sigma}^2$)	$512 \to 512$, k=3, s=1, p=1	$512 \times 8 \times 8$	2,359,808
Decoder	ConvTranspose2d + BN + ReLU	$512 \to 256$, k=4, s=2, p=1	$256 \times 16 \times 16$	2,097,408
	ConvTranspose2d + BN + ReLU	$256 \to 128$, k=4, s=2, p=1	$128 \times 32 \times 32$	524,544
	ConvTranspose2d + BN + ReLU	$128 \to 64$, k=4, s=2, p=1	$64 \times 64 \times 64$	131,200
	ConvTranspose2d + Tanh	$64 \to 3$, k=4, s=2, p=1	$3 \times 128 \times 128$	3,075
Total parameters				**≈10.2M**

Remark 1. *The model size (∼10.2M parameters) is deliberately comparable to the U-Net GAN generator (∼31M parameters) and smaller than the LaMa generator (∼27M parameters with 18*

FFC ResNet blocks), ensuring that any performance differences reflect architectural philosophy rather than simply model capacity.

5 Training Objective: Hybrid VAE–GAN Loss

Pure VAE training with only the ELBO often produces blurry outputs because the pixel-wise reconstruction loss (MSE or L1) averages over the space of plausible completions. To address this while retaining the VAE's principled latent space, we propose a **hybrid VAE–GAN objective** that combines the ELBO with the adversarial and perceptual losses already present in the FlowerPower codebase.

5.1 Loss Components

The total generator loss is:

$$\boxed{\mathcal{L}_{\text{total}} = \lambda_{\text{kl}}\,\mathcal{L}_{\text{KL}} + \lambda_{\text{L1}}\,\mathcal{L}_{\text{L1}} + \mathcal{L}_{\text{adv}} + \lambda_{\text{edge}}\,\mathcal{L}_{\text{edge}} + \lambda_{\text{perc}}\,\mathcal{L}_{\text{perc}}} \tag{8}$$

Each component is detailed below.

5.1.1 KL Divergence Loss (VAE-Specific)

For a spatial latent map of shape $C \times H' \times W'$:

$$\mathcal{L}_{\text{KL}} = \frac{1}{C \cdot H' \cdot W'} \sum_{i} \frac{1}{2}\Big(-1 - \log \sigma_i^2 + \mu_i^2 + \sigma_i^2 \Big) \tag{9}$$

This is the closed-form KL divergence between the learned Gaussian posterior and the standard normal prior, averaged over all spatial positions and channels.

5.1.2 L1 Reconstruction Loss

Pixel-level accuracy on the masked region, identical to the existing GAN and LaMa implementations:

$$\mathcal{L}_{\text{L1}} = \big\| (\hat{\mathbf{x}} - \mathbf{x}_{\text{real}}) \odot \mathbf{m} \big\|_1 \tag{10}$$

5.1.3 Adversarial Loss

Binary cross-entropy loss using the shared `Discriminator`:

$$\mathcal{L}_{\text{adv}} = -\log D(\mathbf{x}_{\text{comp}}) \tag{11}$$

5.1.4 Gradient/Edge Loss

From `shared_utils.compute_gradient_loss`:

$$\mathcal{L}_{\text{edge}} = \big\| \nabla_x \mathbf{x}_{\text{comp}} - \nabla_x \mathbf{x}_{\text{real}} \big\|_1 + \big\| \nabla_y \mathbf{x}_{\text{comp}} - \nabla_y \mathbf{x}_{\text{real}} \big\|_1 \tag{12}$$

5.1.5 VGG Perceptual Loss (Optional)

From the `VGGPerceptualLoss` class in `fine_tune_lama.py`:

$$\mathcal{L}_{\text{perc}} = \sum_{\ell \in \{1,2,3\}} \left\| \phi_\ell(\mathbf{x}_{\text{comp}}) - \phi_\ell(\mathbf{x}_{\text{real}}) \right\|_1 \tag{13}$$

where ϕ_ℓ denotes the feature maps at layer ℓ of a frozen VGG-16.

5.2 Recommended Hyperparameters

Table 6: Recommended loss weights for the hybrid VAE–GAN training.

Loss Component	Weight	Rationale
λ_{kl}	$0.0 \to 0.5$ (annealed)	Start at 0, linearly ramp over 5 epochs
λ_{L1}	100	Consistent with existing GAN training
$\mathcal{L}_{\text{adv}}$	1.0	Standard adversarial weight
λ_{edge}	50	Consistent with existing GAN training
λ_{perc}	10	Consistent with LaMa fine-tuning

6 Training Challenges and Mitigation Strategies

VAE training for high-resolution image generation presents several well-known challenges. We discuss each and the corresponding mitigation strategy.

6.1 Posterior Collapse

Definition 1 (Posterior Collapse)**.** *Posterior collapse occurs when the approximate posterior* $q_\phi(\mathbf{z} \mid \mathbf{x})$ *collapses to the prior* $p(\mathbf{z})$*, i.e.,* $\text{KL}(q_\phi \| p) \approx 0$*, meaning the latent code carries no information about the input and the decoder learns to ignore* $\mathbf{z}$ *entirely.*

This is the most critical failure mode for VAEs. If the KL term dominates early in training before the decoder has learned to use the latent code, the encoder learns to match the prior exactly, and the decoder learns to produce the mean output regardless of $\mathbf{z}$.

6.1.1 Mitigation: KL Annealing

KL annealing [5] gradually increases the weight of the KL term during training:

$$\lambda_{\text{kl}}(t) = \min\left(\lambda_{\text{kl}}^{\max},\ \frac{t}{T_{\text{warmup}}} \cdot \lambda_{\text{kl}}^{\max}\right) \tag{14}$$

where t is the current epoch and T_{warmup} is the warmup period.

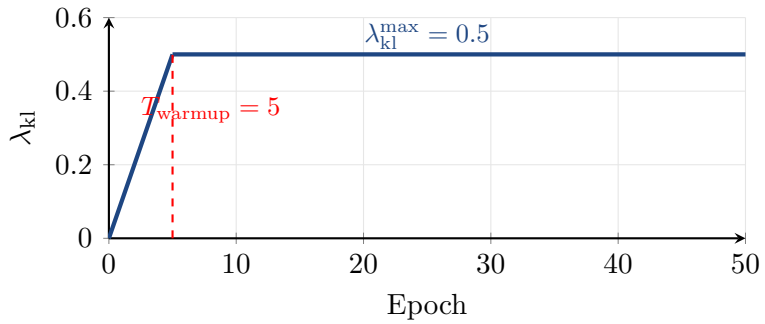

Figure 2: KL annealing schedule: the KL weight increases linearly from 0 to $\lambda_{\text{kl}}^{\max}$ over the first T_{warmup} epochs, then remains constant.

6.1.2 Mitigation: Free Bits

An alternative (or complement) to annealing is the **free bits** strategy [6], which sets a minimum threshold λ_{fb} for the per-dimension KL:

$$\mathcal{L}_{\text{KL}}^{\text{fb}} = \sum_i \max\Big(\lambda_{\text{fb}},\ \text{KL}_i(q_\phi \| p)\Big) \tag{15}$$

This ensures each latent dimension encodes at least λ_{fb} nats of information, preventing collapse while still regularising dimensions that carry excessive information.

6.2 Blurriness

Standard VAE reconstruction losses (MSE, L1) produce blurry outputs because they optimise the expected pixel value, which averages over the multimodal distribution of plausible completions.

Key Insight

The hybrid VAE–GAN objective (Section 5) directly addresses blurriness through three complementary mechanisms:

1. **Adversarial loss**: The discriminator penalises blurry outputs as "fake."
2. **Gradient/edge loss**: Explicitly penalises loss of high-frequency detail.
3. **VGG perceptual loss**: Enforces feature-level similarity rather than pixel-level averaging.

These are the same tools already proven effective in the project's GAN and LaMa training.

6.3 Training Stability of Hybrid VAE–GAN

Combining VAE and GAN objectives introduces a multi-objective optimisation problem. Empirical best practices include:

1. **Separate learning rates**: Use a lower learning rate for the generator/encoder (10^{-4}) and a standard rate for the discriminator (2×10^{-4}).

2. **Delayed discriminator start**: Optionally train the VAE with only reconstruction + KL for the first 1–2 epochs before introducing the adversarial loss.

3. **Gradient clipping**: Clip generator gradients to prevent KL or adversarial loss spikes from destabilising training.

7 Integration with Existing Codebase

7.1 Proposed Directory Structure

```
FlowerPower/
  shared_utils.py            # Unchanged (ImageDataset, Discriminator,
     ...)
  GAN_implementation/        # Existing U-Net GAN
  lama_finetuning/           # Existing LaMa fine-tuning
  vae_implementation/        # NEW
      main.py                # VAE-GAN training loop
      evaluate.py            # Evaluation script (mirrors GAN version)
      Colab_Training.ipynb   # Notebook for Colab/Kaggle training
```

7.2 Code Reuse from `shared_utils.py`

All four shared components are reused without modification:

Table 7: Shared utility reuse in the proposed VAE implementation.

Utility	Used In	Purpose in VAE Training
`ImageDataset`	Data loading	Identical image loading pipeline
`Discriminator`	Adversarial training	Provides GAN loss for sharpness
`get_large_random_mask`	Mask generation	Same mask strategies
`compute_gradient_loss`	Edge loss	Penalises blurriness

Additionally, the `VGGPerceptualLoss` class from `fine_tune_lama.py` should be moved to `shared_utils.py` to enable reuse across all three approaches.

7.3 Training Loop Pseudocode

Algorithm 1 VAE–GAN Training Loop for Inpainting

for epoch $= 0$ **to** num_epochs **do**
  Update λ_{kl} via annealing schedule (14)
  for each batch of real images **do**
    Generate random masks via `get_large_random_mask`
    Compute masked images: $\mathbf{x}_{\text{masked}} = \mathbf{x}_{\text{real}} \odot (1 - \mathbf{m})$
    Compose generator input: $\mathbf{g}_{\text{in}} = [\mathbf{x}_{\text{masked}} \oplus \mathbf{m}]$
    — **VAE Forward Pass** —
    Encode: $\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2 = \text{Encoder}(\mathbf{g}_{\text{in}})$
    Sample: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$
    Decode: $\hat{\mathbf{x}} = \text{Decoder}(\mathbf{z})$
    Composite: $\mathbf{x}_{\text{comp}} = \mathbf{x}_{\text{masked}} + \hat{\mathbf{x}} \odot \mathbf{m}$
    — **Discriminator Update** —
    $\mathcal{L}_D = \text{BCE}(D(\mathbf{x}_{\text{real}}), 1) + \text{BCE}(D(\mathbf{x}_{\text{comp}}.\text{detach}()), 0)$
    Update D with $\nabla \mathcal{L}_D$
    — **Generator (VAE) Update** —
    $\mathcal{L}_{\text{KL}}$, $\mathcal{L}_{\text{L1}}$, $\mathcal{L}_{\text{adv}}$, $\mathcal{L}_{\text{edge}}$, $\mathcal{L}_{\text{perc}}$ as in (9)–(13)
    $\mathcal{L}_{\text{total}} = \lambda_{\text{kl}}\mathcal{L}_{\text{KL}} + \lambda_{\text{L1}}\mathcal{L}_{\text{L1}} + \mathcal{L}_{\text{adv}} + \lambda_{\text{edge}}\mathcal{L}_{\text{edge}} + \lambda_{\text{perc}}\mathcal{L}_{\text{perc}}$
    Update VAE (Encoder + Decoder) with $\nabla \mathcal{L}_{\text{total}}$
  end for
  Save checkpoints
end for

8 Expected Three-Way Comparison Framework

With the addition of the VAE, the project enables a principled comparison across three generative paradigms:

Table 8: Three-way comparison of generative approaches for flower inpainting.

Property	U-Net GAN	LaMa (FFC)	VAE–GAN (Proposed)
Generator	U-Net with skip connections	FFCResNetGenerator (18 blocks)	Conv encoder–decoder with latent bottleneck
Receptive field	Local (conv kernels)	Global (Fourier domain)	Local + bottleneck compression
Training signal	Adversarial + L1 + Edge	Adversarial + L1 + Edge + Perceptual	**ELBO** + Adversarial + L1 + Edge + Perceptual
Latent space	None (deterministic)	None (deterministic)	**Structured Gaussian**
Stochastic outputs	No	No	**Yes** (sample multiple completions)
Pre-trained?	No (from scratch)	Yes (big-lama weights)	No (from scratch)
Parameters	∼31M	∼27M	∼10M

8.1 Evaluation Metrics

For a fair comparison, all three models should be evaluated with the same metrics on the same held-out test set:

1. **L1 / L2 Error** (masked region): Pixel-level accuracy of the inpainted region.
2. **PSNR**: Peak Signal-to-Noise Ratio of the composite image.
3. **SSIM**: Structural Similarity Index capturing perceptual quality.
4. **LPIPS**: Learned Perceptual Image Patch Similarity (using a pre-trained VGG or AlexNet backbone), which correlates better with human judgement than pixel-based metrics.
5. **FID**: Fréchet Inception Distance on the composite images, measuring distributional similarity to real images.

Note

The VAE uniquely enables a **diversity metric**: by sampling multiple latent codes for the same masked input, one can measure the variance across completions, demonstrating the model's ability to represent the multimodal distribution of plausible inpaintings. This is a distinctive advantage over the deterministic GAN and LaMa approaches.

9 Conclusion and Next Steps

This research document has established the theoretical and practical foundations for introducing a VAE-based approach to the FlowerPower inpainting project. Our key findings and recommendations are:

1. **VAEAC** is the most relevant existing model but requires significant architectural re-engineering for 128×128 convolutional inputs. It remains a valuable academic reference point.
2. A **custom Convolutional VAE–GAN hybrid** is the recommended approach, offering:
   - Direct integration with the existing `shared_utils.py` infrastructure,
   - A fair comparison by using identical data pipelines, mask strategies, and discriminators,
   - A distinctive probabilistic perspective (structured latent space, stochastic outputs) that complements the deterministic GAN and LaMa approaches.
3. **KL annealing** is essential to prevent posterior collapse. We recommend a linear warmup over 5 epochs from $\lambda_{\text{kl}} = 0$ to $\lambda_{\text{kl}} = 0.5$.
4. The **hybrid loss** (ELBO + adversarial + edge + optional perceptual) addresses the inherent blurriness of pure VAEs while preserving the principled latent space.

Recommended Implementation Order

1. Implement `ConvVAEGenerator` module with encoder, reparameterisation, and decoder.
2. Write the training loop in `vae_implementation/main.py` with KL annealing.
3. Train for 50 epochs (matching the U-Net GAN configuration).
4. Adapt `evaluate.py` for VAE-specific evaluation (including multi-sample diversity).
5. Run the three-way comparison and compile results.

References

[1] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *Proc. ICLR*, 2014.

[2] D. J. Rezende, S. Mohamed, and D. Wierstra, "Stochastic Backpropagation and Approximate Inference in Deep Generative Models," in *Proc. ICML*, pp. 1278–1286, 2014.

[3] K. Sohn, H. Lee, and X. Yan, "Learning Structured Output Representation using Deep Conditional Generative Models," in *Proc. NeurIPS*, pp. 3483–3491, 2015.

[4] O. Ivanov, M. Figurnov, and D. Vetrov, "Variational Autoencoder with Arbitrary Conditioning," in *Proc. ICLR*, 2019.

[5] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating Sentences from a Continuous Space," in *Proc. CoNLL*, pp. 10–21, 2016.

[6] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, "Improved Variational Inference with Inverse Autoregressive Flow," in *Proc. NeurIPS*, pp. 4743–4751, 2016.

[7] A. B. L. Larsen, S. K. Sønderby, H. Larochelle, and O. Winther, "Autoencoding beyond pixels using a learned similarity metric," in *Proc. ICML*, pp. 1558–1566, 2016.

[8] R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Gusarev, D. Kalinin, and A. Babenko, "Resolution-robust Large Mask Inpainting with Fourier Convolutions," in *Proc. WACV*, pp. 2149–2159, 2022.

[9] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets," in *Proc. NeurIPS*, pp. 2672–2680, 2014.

[10] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. MICCAI*, pp. 234–241, 2015.

[11] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *Proc. ICLR*, 2015.

[12] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *Proc. ECCV*, pp. 694–711, 2016.

[13] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *Proc. NeurIPS*, pp. 6626–6637, 2017.

[14] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *Proc. CVPR*, pp. 586–595, 2018.

[15] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in *Proc. CVPR*, pp. 10684–10695, 2022.